Refine Minimum Releasable Features (MRFs)

As I described in Chapter 17, the minimum releasable features represent the smallest set of "must-have" features, the ones that simply have to be in the release if we are to meet customer value and quality expectations. An important part of release planning is to diligently reevaluate and refine what are truly the MRFs for the release. As we obtain fast feedback from our sprints and acquire validated learning, we are constantly adjusting the MRFs.

A problem I frequently see in organizations is an inability to agree on what constitutes the MRFs. Multiple competing stakeholders simply might not agree. Having poorly defined MRFs or MRFs that people only passive-aggressively agree with interferes with clear decision making during release planning. For example, we're running out of time; which feature should we drop? Lack of clarity regarding the MRFs might complicate this decision.

In Scrum, the product owner is ultimately responsible for defining the MRFs. Of course, he can and will do this in close collaboration with the proper stakeholders and the Scrum team.

For some, the MRFs concept may feel counterintuitive—why not try to deliver the largest set of features in a release instead of the smallest? The simple answer is that the largest set of features probably costs the most money, takes the most time, and has the most risk. Conversely, the smallest possible feature set should cost the least money, take the least time, and have the least risk. Thinking minimally better aligns us with the principle of delivering smaller and more frequent releases, as described in Chapter 14.

The MRFs should be defined with knowledge of the feature sizes, as determined during product backlog grooming. Not everyone agrees. Some believe that the MRFs should be defined independently of cost—meaning the MRFs are the minimum releasable features that will meet the users' value threshold for this release (and this determination is made independently of cost data). Initial MRFs can be envisioned without cost data, but because all of our release-planning decisions need to be made within a sensible economic framework, knowing feature costs provides a critical check on the economic viability of the MRFs. If we determine that the MRFs are not economically viable, perhaps it is time to pivot.

Sprint Mapping (PBI Slotting)

In each sprint the team works on a set of product backlog items. The team and product owner don't decide which specific product backlog items to work on in a given sprint until sprint planning. Does that mean we should give no consideration to mapping product backlog items to sprints before sprint-planning meetings?

Absolutely not! Some teams believe that a quick, early mapping (or slotting) of near-term product backlog items into sprints is helpful. For example, mapping out a

few sprints in a multiteam environment might help the teams better coordinate their work.

To do this mapping we need an appropriately detailed, estimated, and prioritized product backlog. Using our team's velocity, we can approximate a set of product backlog items for each sprint by grouping together items whose aggregate size roughly equals the team's average velocity. The result might look like the left side of Figure 18.5. Some people prefer to show the sprint map horizontally (right side of Figure 18.5) to more closely resemble a timeline. I have seen some teams place the horizontally oriented sprint map of a Scrum team above a standard project plan (a Gantt chart, for example) that describes the work of non-Scrum teams to better visualize the alignment and touch points between the Scrum development work and the non-Scrum work.

When developing with one Scrum team we might do this mapping during initial release planning to get a rough idea of when certain features within the release will be created. This mapping activity might also cause us to reorganize the product backlog to group items in ways that are more natural or efficient. We also might choose to

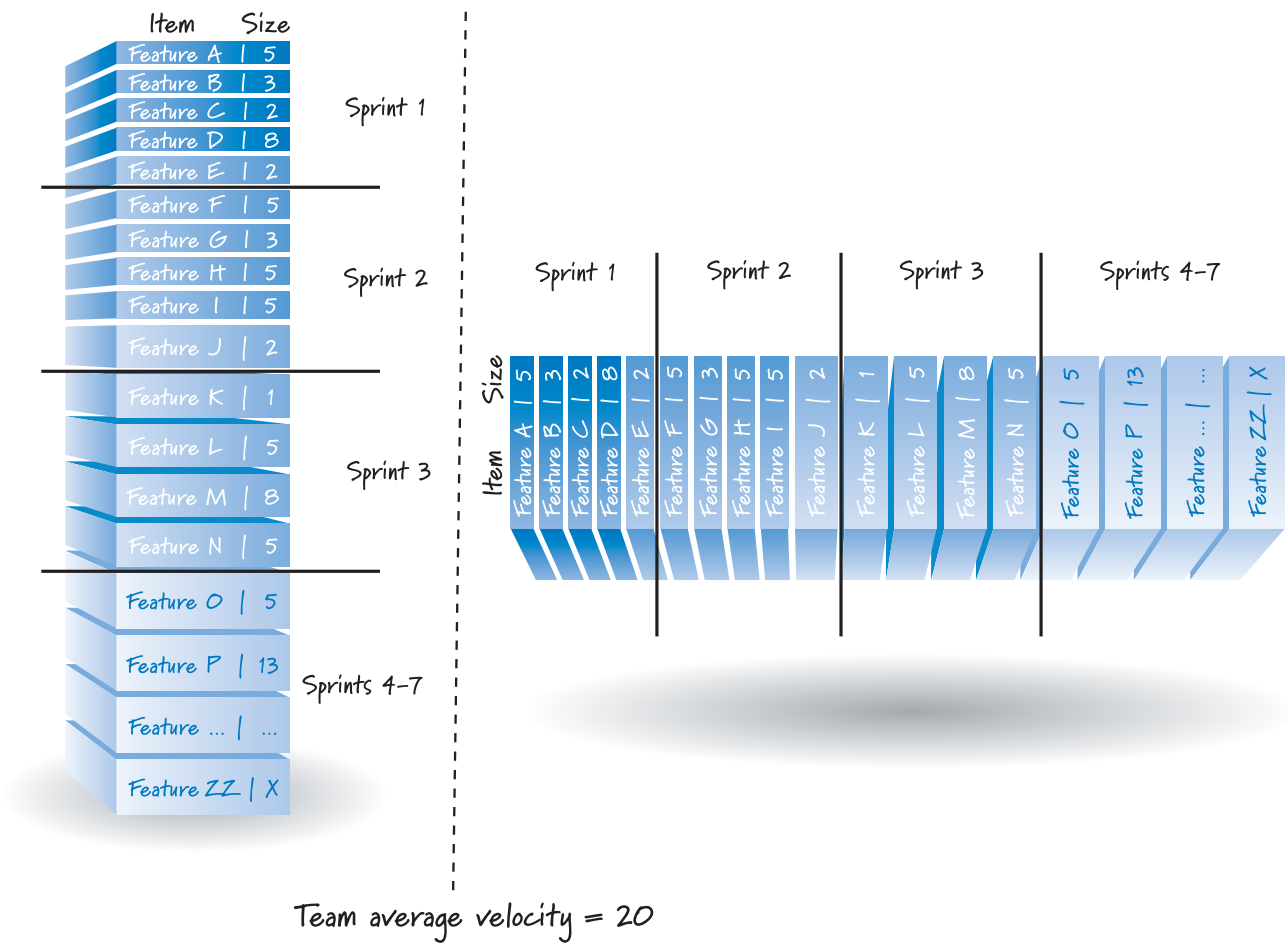


FIGURE 18.5 Mapping product backlog items to sprints

reorganize the work to ensure that the results at the end of a sprint are sufficient for us to get validated learning and actionable feedback.

When developing with multiple teams, we may want to do some forward-looking mapping of items to sprints to help manage inter-team dependencies. One approach is to use a form of rolling look-ahead planning (Cohn 2006), where each team considers the needed backlog items not only for the upcoming sprint but for at least two (and sometimes more) future sprints. This way, when more than one Scrum team is involved, the teams can know what team is going to work on which items and roughly when.

For example, assume that SR4U will have three Scrum teams. Team 1 focuses on the end-to-end processing of user requests. This team enables users to specify a review query, run the query, and get review results. Team 2 focuses on the AI engine, which has the logic for how to analyze and discriminate among reviews. Team 3 focuses on connecting to different Internet data sources for retrieving candidate reviews.

These three teams must coordinate their efforts to make sure that the minimum releasable features are produced and available in time for the Social Media Expo. It makes sense for all three teams to participate in joint release planning.

During an initial release-planning meeting each of the teams provides an idea of when it will work on its product backlog items. In the ensuing discussion, team 1 might say, "We think we'll be ready to create the *ignore reviews with specific keywords* feature in sprint 2. However, we will need team 3 to be able to retrieve data from at least one Internet source either before that sprint begins or very early during sprint 2." Members of team 3 can then examine their sprint map to see if they are currently planning to have at least one source available by sprint 2. If not, the two teams can discuss the dependency and see what modifications one or both of the teams need to make.

If we choose to do some early mapping, we must realize that this mapping can and will evolve during the creation of the release. Ultimately the decision as to which features each team will work on in a given sprint is made at the last responsible moment—the sprint planning for that sprint.

Alternatively, a preference of many organizations (especially those doing product development using only one team) is to perform little or no early mapping of product backlog items to sprints. Teams in such companies believe that the effort to produce the mapping is not justified by the value that it delivers. For these teams, the initial release-planning meeting would not involve the mapping step, at least not in any significant way.

Fixed-Date Release Planning

As I mentioned earlier, many organizations that use Scrum prefer to use fixed-date releases. Table 18.2 summarizes the steps for performing fixed-date release planning.

Let's use SR4U as an example. Release 1.0 is tied to the start of the Social Media Expo, which starts on Monday, September 26. The company decides that having an

TABLE 18.2 Steps for Performing Fixed-Date Release Planning

Step	Description	Comments
1	Determine how many sprints are in this release.	If all sprint lengths are equal, this is simple calendar math because you know when the first sprint will start and you know the delivery date.
2	Groom the product backlog to a sufficient depth by creating, estimating the size of, and prioritizing product backlog items.	Because we are trying to determine which PBIs we can get by a fixed date, we need enough of them to plan out to that date.
3	Measure or estimate the team's velocity as a range.	Determine an average faster and an average slower velocity for the team (see Chapter 7).
4	Multiply the slower velocity by the number of sprints. Count down that number of points into the product backlog and draw a line.	This is the "will-have" line.
5	Multiply the faster velocity by the number of sprints. Count down that number of points into the product backlog and draw a second line.	This is the "might-have" line.

initial version to demonstrate at this conference is an excellent first milestone on the path to realizing the product vision.

Although we just imagined SR4U with three teams, for this example, let's go back to the initial assumption that just one team would develop the product. Because Roger and the team want to begin sprint 1 the first week in July and finish by September 23 (the Friday before the Expo starts), they can easily calculate how many sprints they will need to perform in this release. Assuming that the length of each sprint is the same throughout this release, which is the normal case with Scrum, SR4U Release 1.0 has six, two-week (ten-day) sprints. Figure 18.6 maps these sprints onto the calendar.

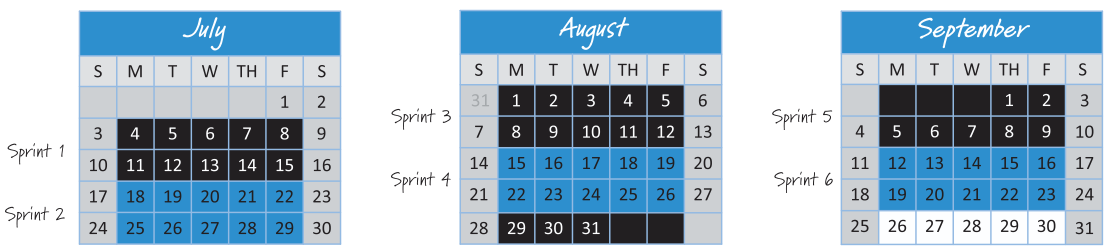


FIGURE 18.6 Sprint calendar for SR4U Release 1.0

Next they determine how much work the team can get done in six sprints. Using the approach I discussed in Chapter 7, let's say they calculate the velocity of the team to be between 18 and 22 story points per sprint. Therefore, Roger and the team should be able to complete between 108 and 132 story points of work during this release.

Now they need to determine what features this range of story points represents. At the end of product planning, Roger and the team had a high-level product backlog with some epic- and theme-level user stories. As I discussed earlier, the SR4U team then conducted a user-story-writing workshop to create more detailed product backlog items. The team then estimated them, and the product owner, with input from the development team and stakeholders, prioritized them.

As part of this process, Roger and the team had to determine the set of must-have features that make up the MRFs. As a rule of thumb, on a fixed-date release I like the MRFs to require less than 100% of the time allocated to the release. I prefer somewhere closer to 60% to 70% for at least two reasons:

- If we run out of time for the release and all of the features targeted for the release are must-have items, which feature should we drop? By definition, if the features are all must-haves, we wouldn't be able to drop any. If we define the release to have about 60% to 70% must-have features, with the remaining scope being nice-to-have features, we can drop nice-to-have features if we have to drop scope.
- If we allocate 100% must-have features to the release, what happens during the release when an emergent must-have feature appears? In other words, a feature we didn't know about earlier presents itself and it absolutely must be included to have a viable release. How would we accommodate it? If we defined the release to include some nice-to-have features, we could drop one or more of them to include the new, emergent must-have feature.

The end result is a product backlog that structurally looks like Figure 18.7. This figure shows the total product backlog as Roger and the team understand it today, including themes and epics that are not planned for this release.

Roger and the team can then apply the results from the earlier velocity calculation, where the team estimated it would be able to complete between 108 and 132 points' worth of work over six sprints. The team can visualize where in the backlog it can get to by counting down a total of 108 points from the top and then counting down to a total of 132 points (see Figure 18.8).

You'll notice that these two lines split the product backlog into three sections (will have, might have, won't have). This approach illustrates how we can give a range answer to the question "What will I get by the release date?" Early in the release it is difficult to give a very precise answer to this question. The range answer is accurate and also communicates the uncertainty we have in the answer—the broader the range, the less certain we are.

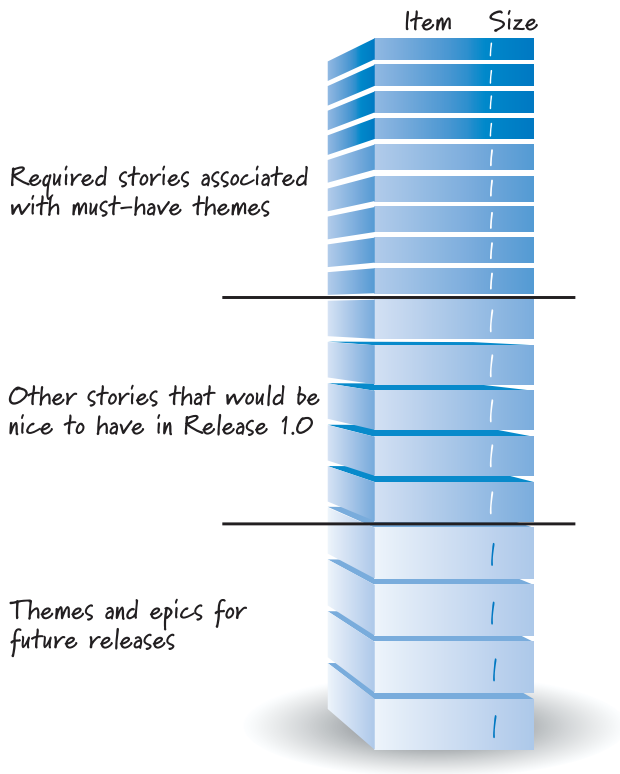


FIGURE 18.7 Product backlog ready for release planning

To understand whether they are in good shape with this release plan, Roger and the team need only to overlay the must-have line (from Figure 18.7) onto the product backlog (from Figure 18.8). Some possible results are shown in Figure 18.9. Notice that the must-have line separates the minimum releasable features that are above the line from the rest of the product backlog.

The left-most product backlog of Figure 18.9 communicates a very positive situation. You can interpret it as "We will have our must-have features." We should proceed with the release.

The middle backlog of Figure 18.9 can be interpreted as "We will have most of our must-have features, but we might or might not have all of them." There is clearly more risk associated with this scenario than the previous scenario. One option is to accept the risk that we won't get all of the must-have items and move on. Because we are planning to learn fast, we might decide to start this release and complete a few sprints. At that point, we could reevaluate where we are and then make a decision to continue with the release or kill it (as I discussed in previous chapters). Also, feedback from the work already completed might indicate that some of the features originally included in the MRFs aren't really must-have items after all and we are actually in good shape.

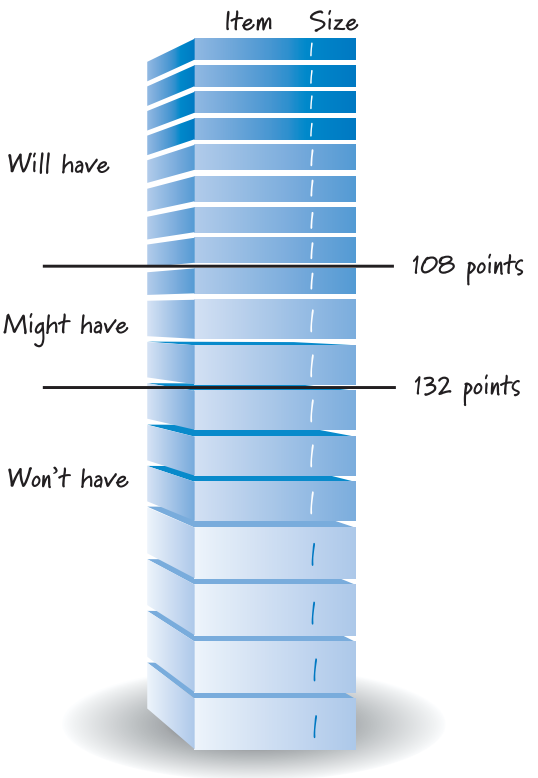


FIGURE 18.8 Determining the range of features on a fixed-date release

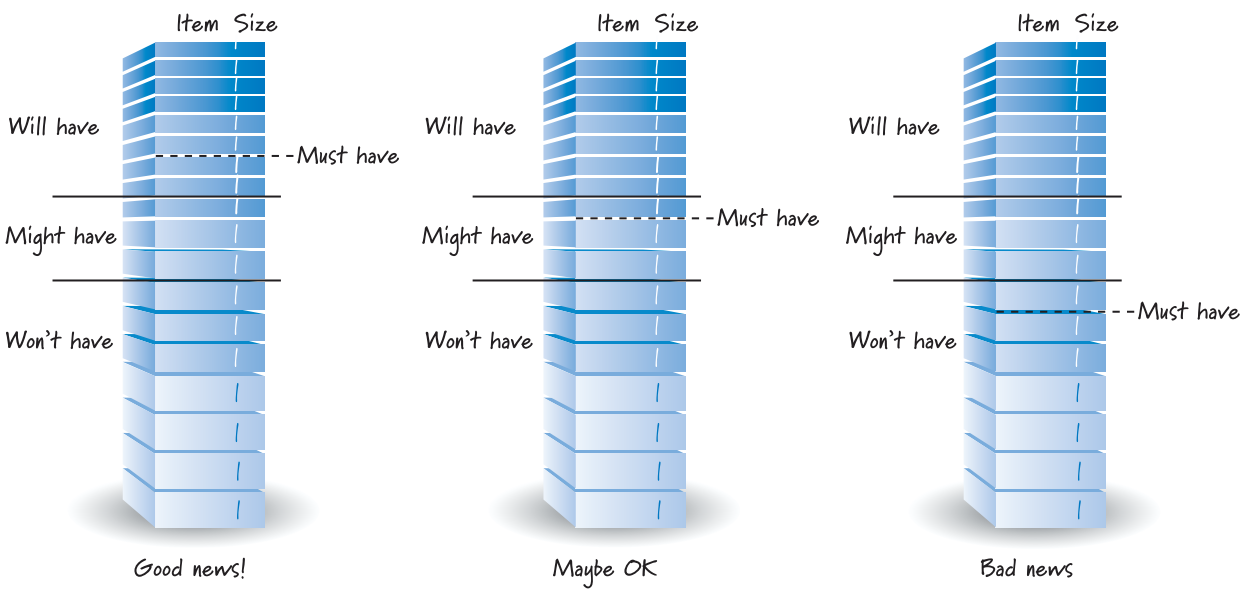


FIGURE 18.9 Location of must-have features relative to the range of deliverable features

Alternatively, we can consider setting a new, later release date that we could easily calculate, or we can suggest adding more people to the development effort to increase velocity (if we think that might help). At this point, some organizations might choose to accrue technical debt by having the team cut corners to ensure that all of the must-have features are delivered by the due date, albeit at a reduced quality level. However, if technical debt is taken on in the current release, it should be paid off in the next release, which will reduce the amount of value delivered then.

The right-most backlog in Figure 18.9 can be interpreted as "We won't have our must-have features." Perhaps we shouldn't proceed with this release, or maybe we should change the release date or consider adding more resources. If we choose to accrue technical debt in this scenario, it will probably be a lot of debt.

Of course, assuming that we proceed with the release, we must revisit our release plan every sprint to update it based on our current knowledge.

For example, at the end of each sprint we have an additional velocity data point, which for a new team without much historical velocity data, or a team doing radically different work from what it is used to, will cause us to recalculate the average points per sprint the team can get done. And, as you might expect, the items in the product backlog could change. New items could emerge and other items could be moved out of this release or deleted altogether as we learn that we don't need them now or ever. To visually communicate the revised release plan, we would redraw a picture similar to Figure 18.8.

Fixed-Scope Release Planning

Although fixed-date releases are very common with Scrum, what if for your product the scope truly is more important than the date? What if you have a large set of must-have features in your minimum releasable features set and you're willing to slip the delivery date to get them all?

If you're in this situation, have you truly winnowed down the must-have features to be the absolute bare minimum? I occasionally hear things like "But we're implementing a standard and you can't ship an implementation of half a standard." While this is perhaps true, in most standards there are still likely to be optional parts that we don't have to implement now (for example, think about web browser support for changing or emerging HTML or CSS standards). In other instances we might be able to go to market with less than the full implementation of the standard and let customers know which parts we support and which parts we don't.

My point is that if we think incrementally and aggressively target the true minimum releasable features, we can usually turn a fixed-scope release into a set of smaller fixed-date releases. When the set of minimum releasable features becomes small, another constraint (like time) typically becomes the dominant constraint.

Let's say that we have winnowed down the must-have features to the bare minimum and our principal release constraint is still focused on when we can we get those features. In this case we perform release planning as outlined in Table 18.3.

If we're doing a fixed-scope release, we must know what the features are at the start of the release. We might know these features if we are building a simple or familiar product. When developing innovative products, however, many features will emerge and evolve during the development effort. We certainly have some idea of the desired features up front, so we'll use them in our initial release planning. However, we must be prepared to continuously revise our release plan as our understanding of the required features changes.

If we perform a release-planning meeting at the start of the release, we must first groom the product backlog as we did during fixed-date planning. A difference is that during fixed-date planning we try to have fewer than 100% must-have items in the release to buffer against uncertainty. In fixed-scope planning we want the entire scope for the release to be must-have features. Our goal on fixed-scope releases is to get all of the must-have features completed in a timely way. If an emergent must-have feature appears, we will simply add it to the scope of the release and push out the release date.

During fixed-date planning, we know precisely how many sprints we will have. During fixed-scope planning, we need to calculate the number of sprints required to deliver the fixed set of features.

TABLE 18.3 Steps for Performing Fixed-Scope Release Planning

Step	Description	Comments
1	Groom the product backlog to include at least the PBIs we would like in this release by creating, estimating the size of, and prioritizing PBIs.	Because this is a fixed-scope release, we need to know which PBIs are in the fixed scope.
2	Determine the total size of the PBIs to be delivered in the release.	If we have a product backlog of estimated items, we simply sum the size estimates of all of the items we want in the release.
3	Measure or estimate the team's velocity as a range.	Determine an average faster and an average slower velocity for the team.
4	Divide the total size of the PBIs by the faster velocity and round up the answer to the next integer.	This will tell us the lowest number of sprints required to deliver the features.
5	Divide the total size of the PBIs by the slower velocity and round up the answer to the next integer.	This will tell us the highest number of sprints required to deliver the features.

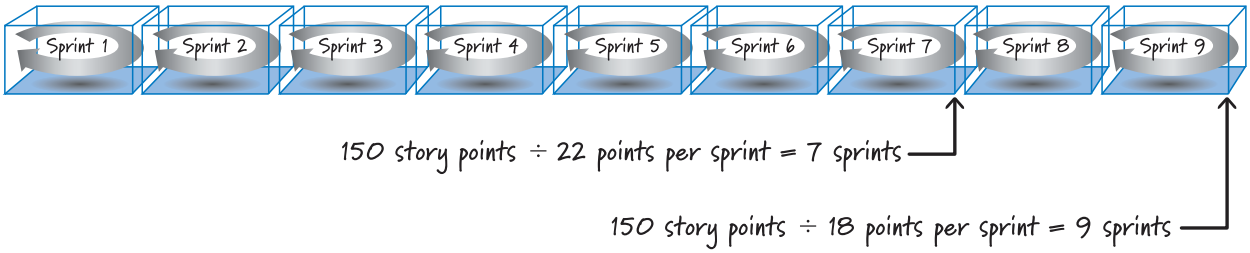


FIGURE 18.10 Results of fixed-scope planning

To perform the math we need the velocity range for our team (as we did with fixed-date planning). Let's say our team's velocity on two-week sprints ranges between 18 and 22 story points. To answer the question of when we will get the fixed set of features, we sum the sizes of all of those features and then divide by our team's higher and lower velocities. The result is a range of sprints within which delivery will take place.

Let's say we want 150 story points of features in the next release. If we divide 150 by 18 (our team's slower velocity) and round up, we get nine sprints. If we divide 150 by 22 (our team's faster velocity) and round up, we get seven sprints. We can visualize this as shown in Figure 18.10.

Notice that once again we give a range answer to the question we are being asked. In this case the question is "How many sprints will you need to complete a release with 150 points of work?" Our answer will be seven to nine sprints. Because these are two-week sprints, our answer also could be stated as 14 to 18 weeks.

Calculating Cost

Calculating costs on either a fixed-date or fixed-scope release is easy (see Table 18.4).

TABLE 18.4 Calculating the Cost of a Release

Step	Description	Comments
1	Determine who is on the team.	Assume that the team composition doesn't materially change either during a sprint or from sprint to sprint.
2	Determine the sprint length.	Assume that all sprints have the same length.
3	Based on team composition and sprint length, determine the personnel costs of running a sprint.	This is simple if previous assumptions are true, and only slightly more complicated if team composition or sprint length fluctuates.

continues

TABLE 18.4 Calculating the Cost of a Release (*Continued*)

Step	Description	Comments
4a	For a fixed-date release, multiply the number of sprints in the release by the cost per sprint.	The result is the fixed personnel cost for this release.
4b	For a fixed-scope release, multiply both the high and low number of sprints by the cost per sprint.	The results are the range of personnel costs associated with the release. One represents the lower amount the release should cost and the other the higher amount it should cost.

Let's assume that the composition of the team assigned to the development effort is reasonably stable. In other words, we aren't taking people off of the team or adding new people to the team. And, if we do, the changes are small and the people we move around get paid something reasonably similar.

Based on these assumptions, we can easily determine the cost per sprint, because we know who is on the team and the length of the sprints. If we remove other costs (like capital costs) from this discussion, which often is reasonable because the majority of software development costs are the cost of the people, personnel cost is a pretty good surrogate for the overall cost per sprint.

To finalize the calculation, we need to know the number of sprints within the release. On a fixed-date release we know exactly the number of sprints, so we multiply the number of sprints by the cost per sprint to determine the release cost.

On a fixed-scope release, we have a range of sprints. In our previous example we calculated a range of seven to nine sprints, so the cost for this release will range from seven to nine times the cost per sprint. Most organizations will budget at the high end of this range because the release may in fact take nine sprints to complete. If we budgeted for only seven sprints, we might have insufficient funds to complete the release.

Another approach to calculating cost can be used if you know your historic cost per story point. If you have data that indicates how many points of work were completed during a previous period of time (say, a year) and you divide that into the loaded labor cost of the team, you will know your cost per point. If it is reasonable to assume that the same cost per point would apply to the current release, you can roughly estimate the cost for a 150-point release (by multiplying 150 by the historic cost per point) even before doing initial release planning.

Communicating

An important aspect of release planning is communicating progress. Although any highly visible way of communicating progress can be used, most teams use some

form of burndown and/or burnup chart as their principal information radiator of release status. Let's look at how to communicate release status on both fixed-scope and fixed-date releases.

Communicating Progress on a Fixed-Scope Release

On a fixed-scope release we have an idea of the total scope of work we wish to achieve. The goal is to communicate how we are progressing toward completing that work.

Fixed-Scope-Release Burndown Chart

A **burndown chart** for a fixed-scope release shows the total amount of unfinished work that remains each sprint to achieve the current release goal. In this type of chart, the vertical axis numbers are in the same units we use to size our product backlog items (typically story points or ideal days). The horizontal axis represents sprints (see Figure 18.11).

Using the example from earlier in the chapter, we have 150 story points at the start of development (at the end of initial release planning), which is the same as the start of sprint 1. At the end of each sprint we update this chart to show the total amount of work remaining within the release. The difference between the amount of work remaining at the beginning of a sprint and the work remaining at the end of the sprint represents the sprint velocity. This is plotted as the "Actual" line in Figure 18.11.

We can also show projected outcomes on the burndown chart. In Figure 18.11 there are three lines predicting when the release might be done, each corresponding to a predicted team velocity. If the team is able to work at its higher velocity of 22 points per sprint, the team would finish by the end of seven sprints. If the team

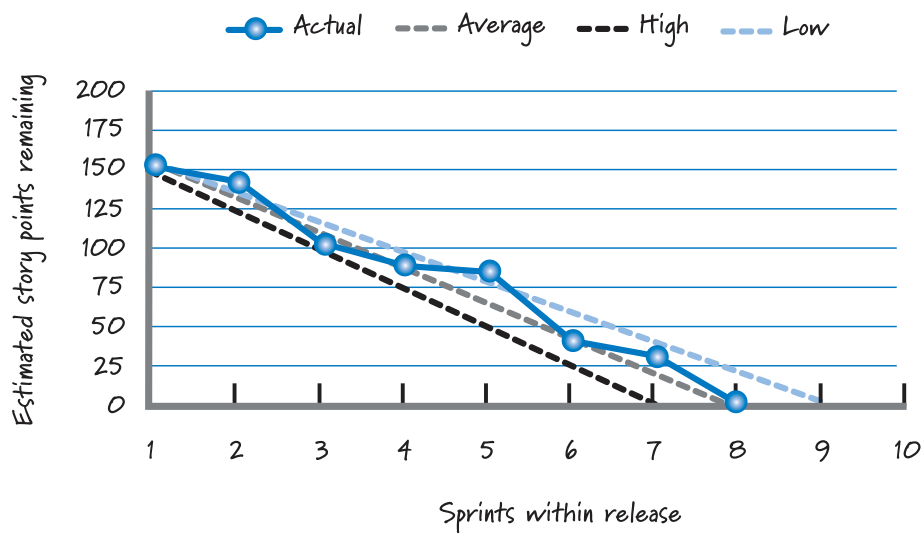


FIGURE 18.11 Fixed-scope-release burndown chart

operates at its lower velocity of 18, it might need a total of nine sprints. And, if the team operates at its average velocity of 20, it would need eight sprints.

There are several variations on the basic burndown chart; however, they are all similar in that they show each sprint the cumulative size of the work remaining to achieve the release goal.

Fixed-Scope-Release Burnup Chart

A **burnup chart** for a fixed-scope release shows the total amount of work in a release as a goal or target line and our progress each sprint toward achieving that goal (see Figure 18.12). The horizontal and vertical dimensions of the chart are identical to those of the release burndown chart.

In this chart, at the end of every sprint we increment the cumulative story points completed by the total story points completed in that sprint. The goal is to burn up to achieve the target number of story points in the release. And, like the release burndown chart, this chart shows the same three predictive lines indicating the likely number of sprints to achieve the target.

Some people prefer to use the burnup format because it can easily show a change in scope for the release. For example, if we add more scope in the current release (so the release really isn't fixed scope!), in the sprint where the new scope is added we simply move the target line up to indicate that a new, higher target exists from this point forward (see Figure 18.13).

It is also possible to show a change in scope on a release burndown chart (see Cohn 2006).

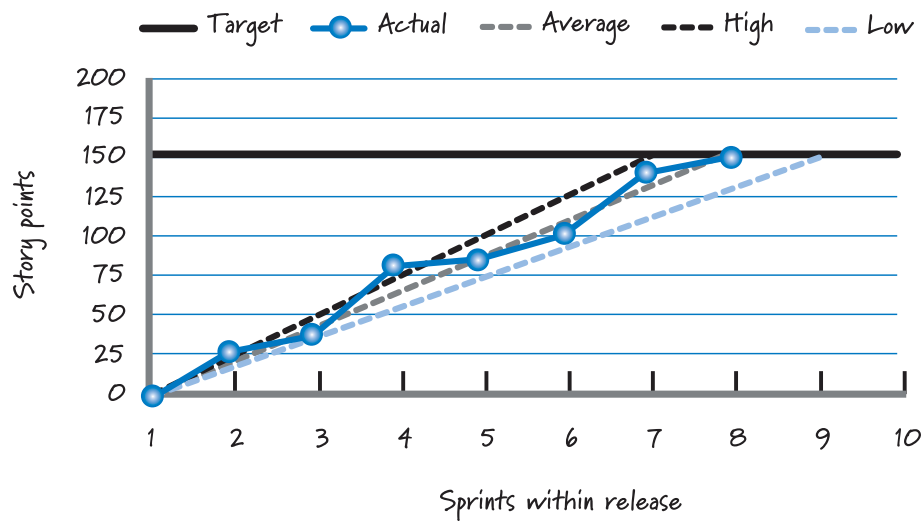


FIGURE 18.12 Fixed-scope-release burnup chart

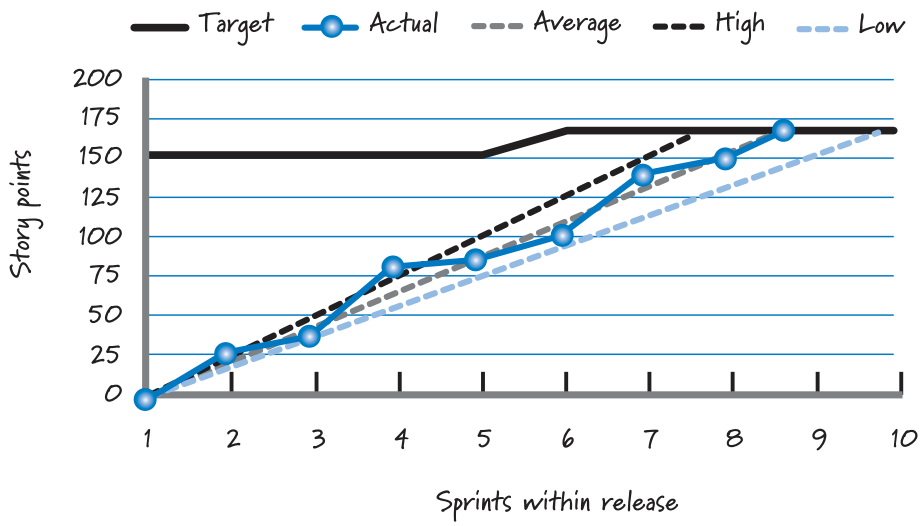


FIGURE 18.13 Variable-scope-release burnup chart

Communicating Progress on a Fixed-Date Release

With a fixed-date release we know the number of sprints in the release, so our goal is to communicate the range of features we expect to complete and our sprint-by-sprint progress toward the range. The traditional burndown and burnup charts aren't effective tools for fixed-date planning because they assume you know how much total scope you have to burn down or burn up. Remember this is fixed-date planning, so we are trying to calculate and communicate over time the narrowing range of scope that can be delivered by a fixed date.

Figure 18.9 shows how to visualize a range of features we expect to achieve in a fixed-date release. If we update the charts in Figure 18.9 at the end of each sprint, we have a very effective way of communicating the projected range of features we will complete by the fixed release date. We will also have an understanding of how likely we are to get the must-have features by the release date.

If we want to maintain one chart that shows our historical progress toward achieving the final scope, we can create a specialized burnup chart in the form of Figure 18.14.

This chart has all of the same elements as the charts in Figure 18.9. However, in Figure 18.14, the product backlog is inverted (intentionally positioned upside down) so that instead of the highest-priority item being physically positioned at the top, it is now at the bottom of the product backlog. Lower-priority items are now found higher up in the backlog. Inverting the backlog in this fashion eliminates the problem of having to know the scope of the product backlog items in the release (which traditional release burndown and burnup charts require).

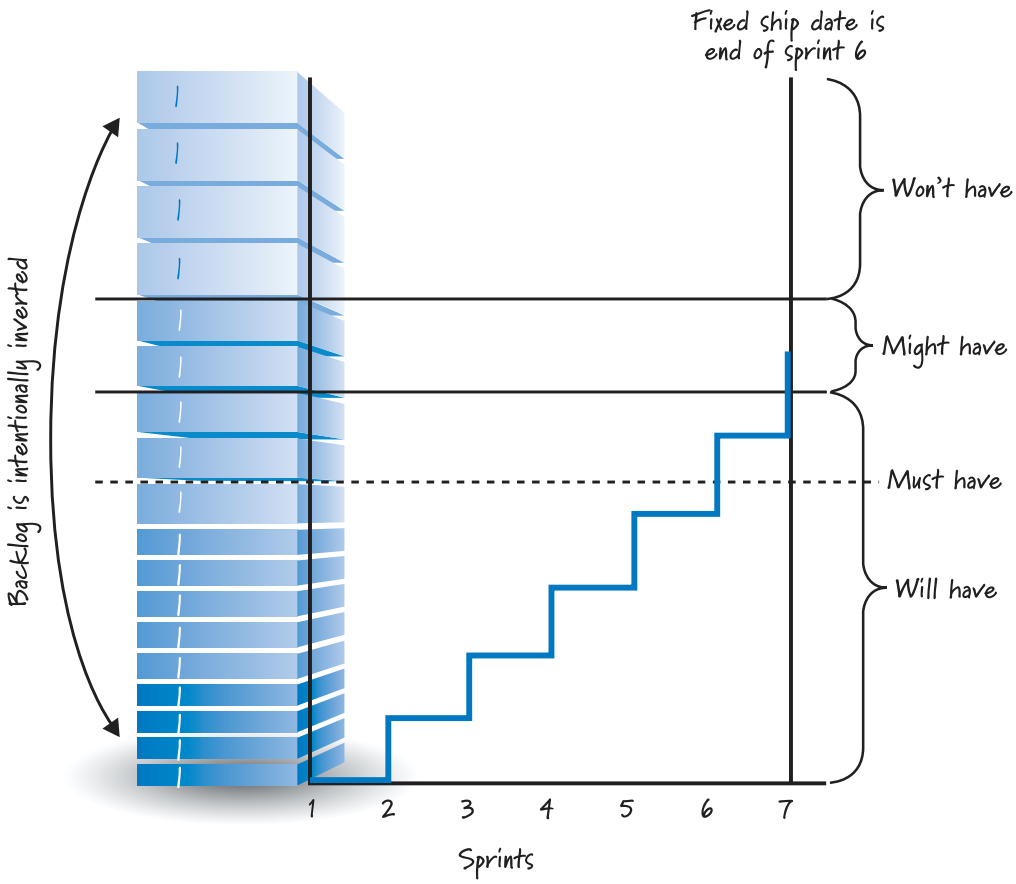


FIGURE 18.14 Fixed-date-release burnup chart (with inverted product backlog)

The chart shows the projected range of features that we expect to have by the end of sprint 6 (beginning of sprint 7). Each sprint we burn up the chart to show the features completed in that sprint. So at the end of sprint 1 (beginning of sprint 2) there is a vertical line whose length indicates how many features we completed in sprint 1. Using this approach allows us to see how we are progressing toward hitting our target range of features, as well as how we are progressing toward completing the must-have features. For simplicity, there are no trend lines on this graph, but they could easily be added to extrapolate from earlier sprints what scope we are likely to end up with.

Closing

In this chapter I expanded the description of release planning by discussing when release planning takes place, who is involved, what activities take place, and the elements of the resulting release plan. I also discussed the details of how to do both

fixed-date and fixed-scope release planning and how to communicate progress during a release.

This chapter concludes Part III. In the next chapter I will discuss the next level of planning: sprint planning, which I include with Part IV to group together all of the chapters related to sprint-specific activities.

This page intentionally left blank

Part IV
Sprinting

This page intentionally left blank

Chapter 19
Sprint Planning

A release is typically composed of multiple sprints, each of which delivers customer or user value. Every sprint begins with sprint planning, a time when the Scrum team gathers to agree on a sprint goal and determine what it can deliver during the forthcoming sprint. In this chapter I discuss where sprint planning fits into the Scrum framework and how to perform it.

Overview

A product backlog may represent many weeks or months of work, which is much more than can be completed in a single, short sprint. To determine the most important subset of product backlog items to build in the next sprint, the Scrum team performs sprint planning. During sprint planning the Scrum team agrees on a goal for the sprint, and the development team determines the specific product backlog items that are aligned with that goal and that it can realistically deliver by the end of the sprint. To acquire confidence in what it can deliver, the development team creates a plan for how to complete the product backlog items. Together the product backlog items and the plan form the sprint backlog.

Timing

Sprint planning is a recurring, just-in-time activity that takes place at the beginning of each sprint, when we can leverage the best possible information to decide what to work on in the upcoming sprint (see Figure 19.1).

For a two-week to month-long sprint, sprint planning should take no longer than four to eight hours to complete.

Participants

The full Scrum team collaborates during sprint planning. The product owner shares the initial sprint goal, presents the prioritized product backlog, and answers any questions the team might have regarding the product backlog items. The development team works diligently to determine what it can deliver and then makes a realistic commitment at the end of sprint planning. The ScrumMaster, acting as the Scrum team coach, observes the planning activity, asks probing questions, and facilitates to help ensure a successful result. Because the ScrumMaster is not in charge of the

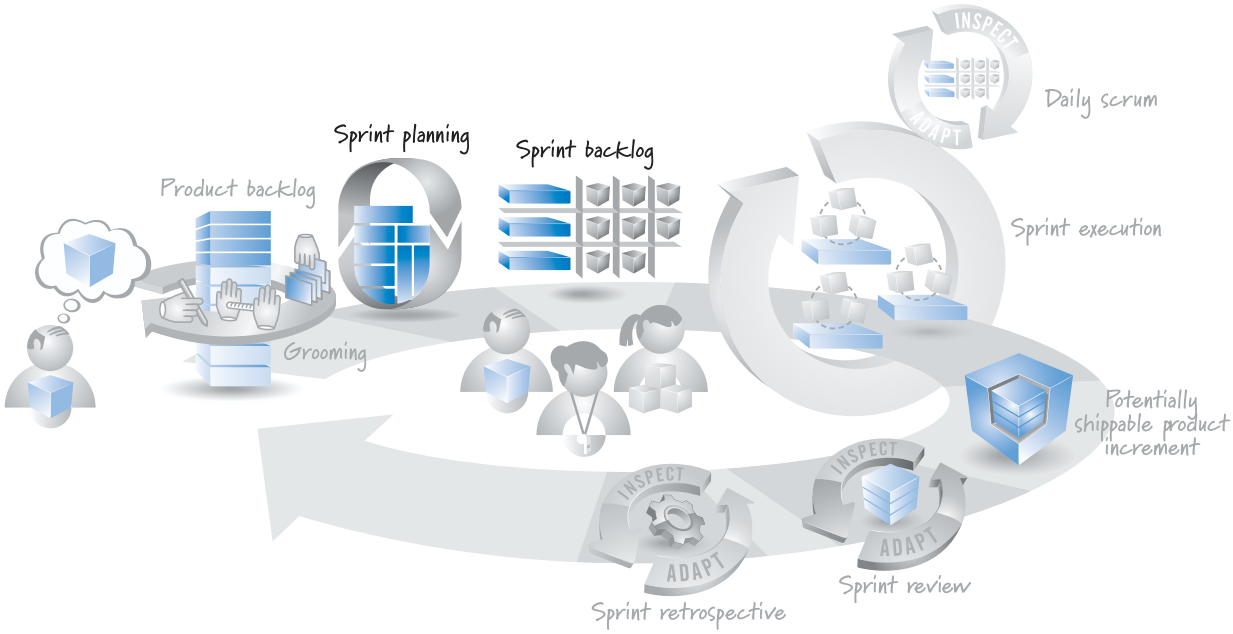


FIGURE 19.1 When sprint planning happens

development team, she cannot decide on behalf of the development team what commitment to make. The ScrumMaster can, however, challenge the team's commitment to ensure that it is realistic and appropriate.

Process

Figure 19.2 illustrates the sprint-planning activity.

Sprint planning relies on a set of inputs that guide the development team in determining what value it can realistically deliver by the end of the sprint. These inputs are described in Table 19.1.

The first and most crucial input to sprint planning is a product backlog that has been groomed prior to sprint planning so that the topmost items meet the Scrum team's definition of ready (see Chapter 6). Typically this means that the topmost items have well-defined acceptance criteria and are appropriately sized, estimated, and prioritized.

Engaged product owners also enter sprint planning having a good idea of what they want the team to deliver by the end of the sprint. They might have a specific set of high-priority product backlog items in mind—"I'd really like to get the top five product backlog items done this sprint"—or they might have a more general notion—"At the end of this sprint I want a typical user to be able to submit a simple keyword query." Knowing the sprint goal helps the team balance competing priorities. A product owner should communicate his initial sprint goal in a way that doesn't unduly influence the development team to commit to more than it realistically can deliver.

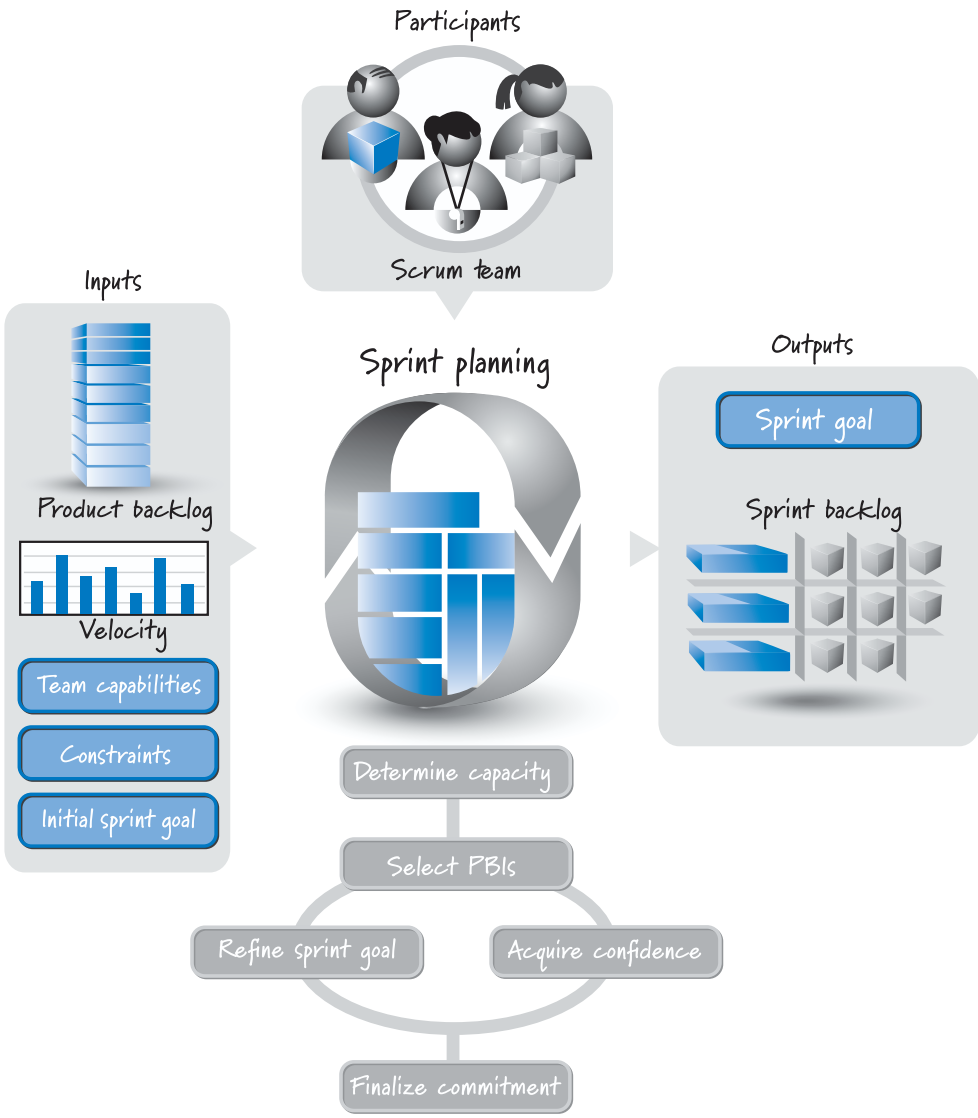


FIGURE 19.2 Sprint-planning activity

TABLE 19.1 Sprint-Planning Inputs

Input	Description
Product backlog	Prior to sprint planning, the topmost product backlog items have been groomed into a *ready* state.
Team velocity	The team's historical velocity is an indicator of how much work is practical for the team to complete in a sprint.

continues

TABLE 19.1 Sprint-Planning Inputs (*Continued*)

Input	Description
Constraints	Business or technical constraints that could materially affect what the team can deliver are identified.
Team capabilities	Capabilities take into account which people are on the team, what skills each team member has, and how available each person will be in the upcoming sprint.
Initial sprint goal	This is the business goal the product owner would like to see accomplished during the sprint.

The fact that the product owner knows what he wants, however, does not necessarily mean that the development team is capable of delivering it during that sprint. A realistic commitment is achieved only through collaboration (and at times negotiation) between the product owner and the development team members. The sprint-planning participants need to have the opportunity to review and discuss potential value-generating alternatives and decide what is practical given the team's capabilities, predicted velocity, and any known constraints.

To acquire confidence in what it can accomplish, the development team will create a plan for how it will achieve the sprint goal. Collectively the selected product backlog items and the plan form the sprint backlog (shown in Figure 19.2). Most teams break down each targeted product backlog item into a set of estimated tasks, which collectively form the plan. Teams that take this approach typically follow a helpful rule of breaking down tasks so that no one task is more than eight hours of effort, although some might be a bit larger. At this level of granularity the team has a good idea of what really needs to be done and whether it can accomplish those tasks in the time it has available.

At the end of sprint planning the development team's commitment is communicated through a finalized sprint goal and the sprint backlog.

Approaches to Sprint Planning

I will describe two approaches to sprint planning: two-part sprint planning and one-part sprint planning.

Two-Part Sprint Planning

One approach to sprint planning is to separate it into two parts (see Figure 19.3). During part 1 (the "what" part) the development team determines its capacity to complete work and then forecasts the product backlog items that it believes it can

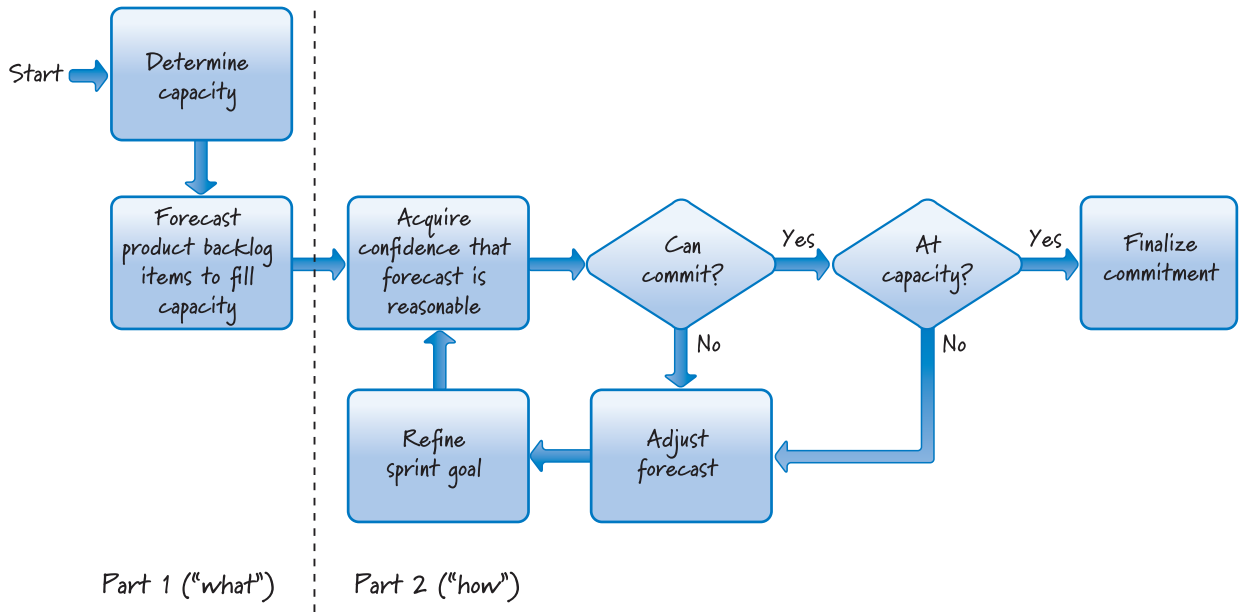


FIGURE 19.3 Two-part sprint-planning approach

deliver by the end of the sprint. So if the team believes it can accomplish 40 story points, it will select about 40 story points' worth of work.

During part 2 (the "how" part) the team acquires confidence in its ability to complete the items that it forecasted in part 1 by creating a plan. Most teams create this plan by breaking the product backlog items into a set of tasks and then estimating (in hours) the effort required to complete each task. The team then compares the estimate of task hours against its capacity, in terms of hours, to see if its initial commitment was realistic.

If the team finds it has selected too much or too little, or has selected items that can't realistically be developed together in the same sprint given one or more constraints, it can adjust its forecast and possibly refine the sprint goal to fit the available capacity and constraints. When the team's forecast is comfortably within its capacity range and constraints, it finalizes its commitment and sprint planning is over.

One-Part Sprint Planning

An alternative approach to sprint planning, and the one I see most frequently, is a one-part approach that interleaves selecting an item and acquiring confidence that it can be delivered. This approach is illustrated in Figure 19.4.

Using this approach, the development team begins by determining its capacity to complete work. Based on available capacity, the sprint goal may need to be refined. Next the team selects a product backlog item and then acquires confidence that the

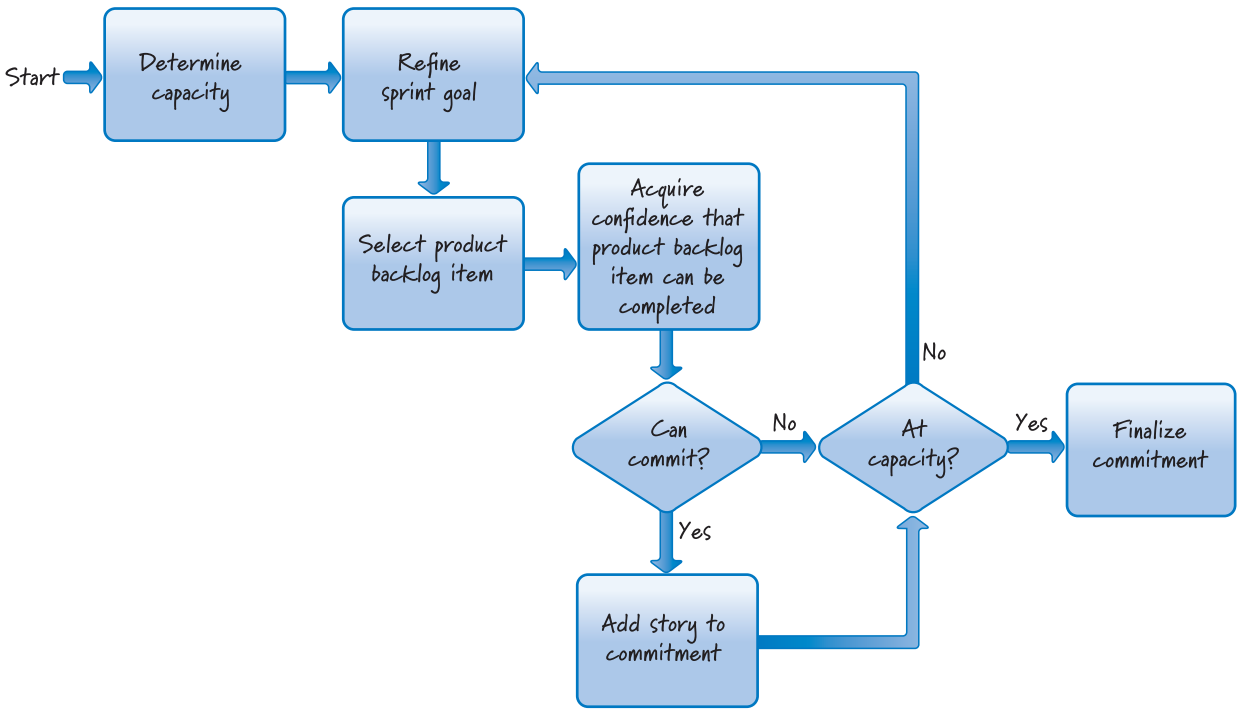


FIGURE 19.4 One-part sprint-planning approach

selected item will reasonably fit within the sprint, given other items already included in the team's evolving commitment. This cycle is then repeated until the team is out of capacity to do any more work. At that point the commitment is finalized and sprint planning is over.

Determining Capacity

An important first activity during sprint planning is determining the available capacity of the team to perform work during the sprint. Knowledge of capacity guides the Scrum team in determining what it can deliver.

What Is Capacity?

Figure 19.5 illustrates the various factors that influence a team's capacity to work on product backlog items during an upcoming sprint. These include the time needed for other Scrum activities, non-sprint-related commitments, personal time off, and the need for a buffer.

Let's say a team is doing a two-week (ten-day) sprint. Right away, we must accept that the team doesn't actually have ten days to dedicate to sprint execution. We know, for instance, that on a two-week sprint about a day of that time needs to be reserved

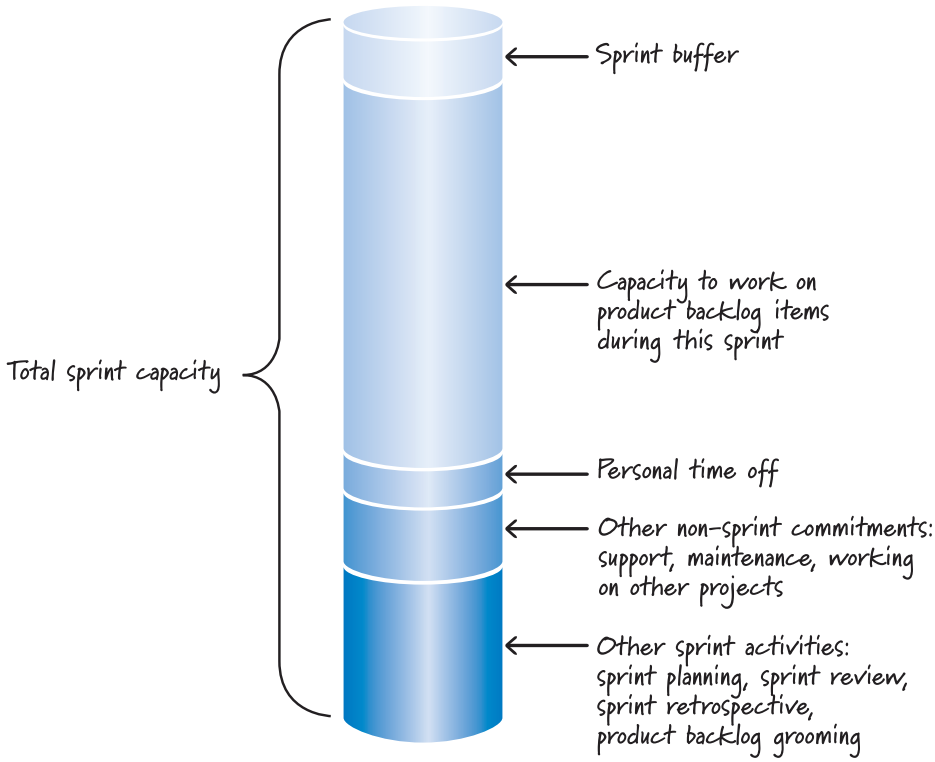


FIGURE 19.5 Development team capacity in a sprint

collectively for sprint-planning, sprint review, and sprint retrospective activities. We also know that the team should reserve up to 10% of its time to assist the product owner with product backlog grooming (writing and refining, estimating, and prioritizing product backlog items) to help ensure that the items are ready.

The team must also determine how much time it should reserve for work outside the sprint, things like supporting the current product, maintaining another product, or other work unrelated to the current sprint. The team should also remember that in an eight-hour day, team members really don't have a full eight hours to work on product backlog items. There is some overhead required to be a good citizen of the organization—attending meetings, responding to emails, interruptions, and so on.

Next, the team needs to know if people have personal time off scheduled during a sprint because that also reduces overall team capacity.

After removing time dedicated to other Scrum activities, work outside the sprint, and personal time off, what remains is the capacity of the team to work on product backlog items for this sprint. However, from this total capacity we should reserve some buffer against things not going quite as planned. For example, any estimating we do won't be perfect, so items might turn out to be a bit larger than we thought. Or something can (and usually does) go wrong. Having a bit of buffer against unexpected problems is wise.

A team can use a number of different approaches to determine a practical buffer size (see Cohn 2006 for some examples). In practice, this buffer can be established empirically after a team performs several sprints and better understands how much buffer should be held in reserve to handle development uncertainty.

Once a buffer is defined, the team can finalize its available capacity for completing work during the sprint.

Capacity in Story Points

What unit of measure should we use for capacity? The two obvious answers are either the same units as the product backlog items (typically story points or ideal days) or the same units as the sprint backlog tasks (effort-hours).

A team's velocity is expressed in terms of product backlog item units (let's say story points). So, if we express capacity in story points, determining capacity is the same as predicting our team's target velocity for the upcoming sprint.

To make this determination, start with the team's long-term average velocity or the team's previous sprint velocity (sometimes referred to as the "yesterday's weather" approach) as an initial estimate of its capacity/velocity for the upcoming sprint. Then consider whether the upcoming sprint might differ from typical or previous sprints (it might not). The result is a reasonable adjusted capacity (predicted velocity) for the upcoming sprint.

For example, let's say our team's average velocity is 40 story points during a two-week sprint. The sprint we are planning, however, occurs during the last two weeks in December in the United States, which means many of our team members will be taking time off for the holidays. We would take on too much work if we used the average velocity of 40; we'd be better off assuming that a velocity closer to 20 (or thereabouts) is a more realistic capacity for the team during this sprint.

Capacity in Effort-Hours

An alternative way to express capacity is in effort-hours. Table 19.2 illustrates how to determine the team's effort-hour capacity to perform task-level work during a two-week or ten-day sprint.

The capacity calculation shown in Table 19.2 is derived as follows. First, the team members express how many days they have available to work on the upcoming sprint (the amount of unavailable time equates to the "personal time off" slice in Figure 19.5). Both Betty and Rajesh are planning to attend a two-day training class, so each of them has only eight days available in the sprint. Simon is planning a three-day weekend so he has nine available days.

Next, the team members determine how much time to reserve for other Scrum activities. They reserve a day of time for sprint planning, sprint review, and sprint retrospective activities. They also deduct the time needed to assist the product owner

TABLE 19.2 Determining Effort-Hour Capacity

Person	Days Available (Less Personal Time)	Days for Other Scrum Activities	Hours per Day	Available Effort-Hours
Jorge	10	2	4–7	32–56
Betty	8	2	5–6	30–36
Rajesh	8	2	4–6	24–36
Simon	9	2	2–3	14–21
Heidi	10	2	5–6	40–48
Total				140–197

with product backlog grooming activities. Together these represent two fewer days available per person to perform task-level work.

The team members then determine how many hours per day they could dedicate to work in this sprint. Each person gives a range that takes into account any overhead work not associated with items in the sprint backlog (the overhead equates to the "other non-sprint commitments" slice in Figure 19.5). For example, Simon is only half-time on this product, so he estimates only two to three hours a day to work on this product during the sprint.

After accounting for personal time off, other Scrum activities, and non-sprint commitments, the team in Table 19.2 estimates a capacity of 140 to 197 effort-hours to work on tasks in the sprint backlog.

I would caution this team against taking on 197 hours of work because it would leave no sprint buffer. A better strategy for this team would be to use a capacity that is probably greater than 140 hours but certainly less than 197 hours when committing to work during this sprint.

Selecting Product Backlog Items

Either approach to sprint planning requires that we select candidate product backlog items for inclusion in the commitment. Selection can be done in several ways. If we have a sprint goal, we would select product backlog items that align with that goal. If there is no formal sprint goal, our default is to select items from the top of the product backlog. We would start with the topmost item and then move to the next item and so forth. If the team were not able to commit to the next-highest-priority item (perhaps there is a skills capacity issue), it would select the next appropriate higher-priority backlog item that looks as if it can be completed within the constraints.

One of my rules when selecting product backlog items for a sprint is that we don't start what we can't finish. So if the next product backlog item is too big to complete in the sprint given the other items that we have already agreed to complete, we should either try to break down the next item into two or more smaller items, each of which would be valuable to our customers, or consider working on another item that we can complete. Also, having a good *definition of ready* will prevent product backlog items from being selected that are poorly defined or have unfulfilled resource or dependency constraints that would prevent our finishing them in a sprint.

The start-only-what-you-can-finish rule is based on the principles that we should limit WIP and that starting something and not finishing it generates a variety of forms of waste. I discussed both of these principles in Chapter 4 when I covered the rule of no goal-altering changes during a sprint. Also, letting incomplete items carry over from one sprint to the next doesn't achieve the goal of having a potentially shippable product increment at the end of each and every sprint.

Acquiring Confidence

One way to acquire confidence is to use predicted velocity to see if the commitment is realistic. If the predicted sprint velocity is 25 story points and our team has selected 45 story points' worth of work, the team should be concerned. At the very least we should start asking questions about why we think the commitment can be achieved. When used this way, velocity provides an excellent check and balance on the proposed commitment.

The risk of using velocity as the sole means of establishing confidence is that even though the numbers look right, the commitment might still be unachievable. For example, if the predicted sprint velocity is 25 story points and the team's commitment totals 21 story points, the commitment would seem reasonable. However, until we dig a little deeper to the task level, we don't really know if the set of product backlog items that total 21 story points can actually be completed—there could be dependency issues, skills capacity issues, as well as a host of other issues that make it impractical for the team to get them all done.

Most Scrum teams gain the necessary level of confidence by breaking the product backlog items down into the tasks that are required to complete them to the Scrum team's agreed-upon definition of done. These tasks can then be estimated (usually in effort-hours) and subtracted from the team's capacity. Breaking product backlog items into tasks is a form of design and just-in-time planning for how to get the items done.

The result is the sprint backlog. Figure 19.6 illustrates an example sprint backlog showing four product backlog items (totaling 21 points) that the team believes it can get done at the end of the sprint. The sprint backlog also shows a plan (in the form of tasks) for delivering the product backlog items to meet the sprint goal.

Is the team making a good commitment or not?

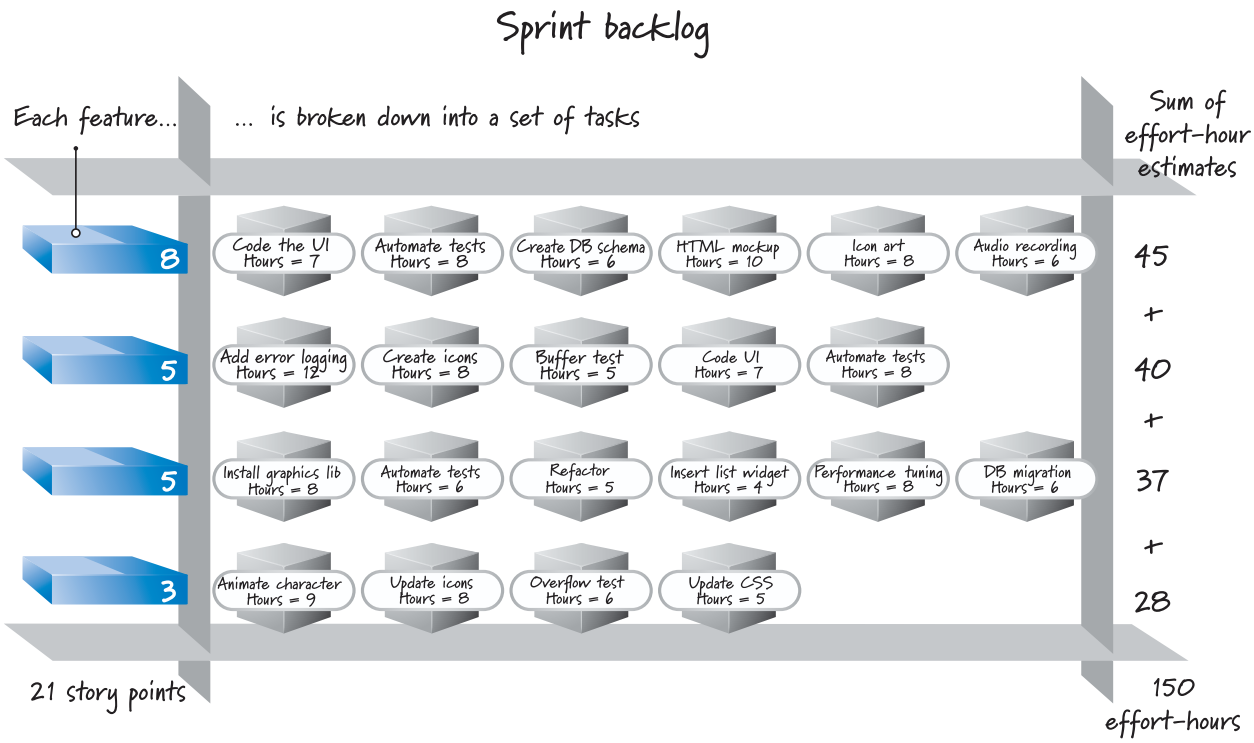


FIGURE 19.6 Sprint backlog showing PBIs and task plan

If the team's predicted velocity is 25 points, a commitment of 21 points seems reasonable. But let's use the task-level details to see if the commitment still looks good. The sum of the tasks for all four product backlog items is 150 effort-hours. Assume that the team for this sprint is the team identified in Table 19.2, which has a task-hour capacity of 140 to 197 effort-hours. A commitment of 150 effort-hours seems comfortably safe—there is likely still a reasonable sprint buffer for handling things that might go wrong.

However, just because the 150 hours are comfortably in the range of 140 to 197 hours doesn't guarantee that the commitment is good. Recall from Table 19.2 that Simon is available only two to three hours a day for nine of the ten days of the sprint. That means Simon has only between 14 and 21 effort-hours available for work on this sprint. What if Simon is the only person who can work on UI tasks? In this case the team might not be able to commit to the four product backlog items in Figure 19.6 because it might run out of "UI capacity" when it runs out of "Simon capacity."

Even though in Scrum we typically don't assign team members to tasks during sprint planning (notice that in Figure 19.6 there are no team member names on the tasks), we need to at least quickly consider our skills capacity or we could make a bad commitment. Just because our commitment is comfortably within the range of estimated aggregate capacity doesn't mean we won't run out of capacity in a particular skills area.